# KEYLOGGER

A Project Report Submitted

By

SAVIO JOHNY (RegNo: 23UBC249)

To

Marian College Kuttikkanam Autonomous

In partial fulfilment of the requirement for the award of the degree of

Bachelor of Computer Applications

Mahatma Gandhi University Kottayam

Department of Computer Applications

Marian College Kuttikkanam Autonomous

Peermade – 685531

February 2026

# DECLARATION

I, SAVIO JOHNY [Reg.No : 23UBC249] certify that the main project report entitled **''Keylogger"** is an authentic work carried out by me at Marian College Kuttikkanam Autonomous. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Student

SAVIO JOHNY

[23UBC249]

Date:

# CERTIFICATE

This is to certify that the project report entitled **"KEYLOGGER"** submitted by **SAVIO JOHNY (Reg. No: 23UBC249)** to **Marian College Kuttikkanam Autonomous** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications** of **Mahatma Gandhi University, Kottayam**, is a bonafide record of the work carried out under my supervision and guidance.


**Ms. Sheela S**
*Guide*
Department of Computer Applications
Marian College Kuttikkanam
Autonomous

**Dr. Rajimol A.**
*Head of the Department*
Department of Computer Applications
Marian College Kuttikkanam
Autonomous


**Examiners**

*Name and Signature*

*Name and Signature*

# ACKNOWLEDGEMENT

"Gratitude is a feeling which is more eloquent than words, more silent than silence." In undertaking this project, we needed the direction, assistance and cooperation of various individuals and organizations, which is received in abundance with grace of God, without their unconstrained support, the project could not have been completed. If words are considered as the symbol of approval and token of acknowledgment, then let the following words play the heralding role of expressing my gratitude. I wish to acknowledge our sincere gratitude to our Principal Prof. Dr Ajimon George, Marian College Kuttikkanam Autonomous, for all their efforts and administration in educating me in this premier institution. I extend our gratitude to Dr Rajimol A, Head of the Department of Computer Applications, who is a constant source of inspiration and whose advice helped me to complete this project successfully. I express my deep sense of gratitude to my internal project guide, Ms. Sheela S, for his guidance for the successful completion of this project. I am also thankful to Mr. Alan Jolly and Mr. Hareesh Rajendran for his efforts in coordinating the project and guiding me through all the phases of the project. With great enthusiasm we express my gratitude to all the faculty members of Department of Computer Applications for their timely help and support. Finally, I express my deep appreciation to all our friends and family members for the moral support and encouragement they have given to complete this project successfully.

# ABSTRACT

Keylogger is an advanced keystroke monitoring application developed to demonstrate the principles of system-level input monitoring and automated remote reporting. In the modern cybersecurity landscape, understanding how surveillance tools function is essential for building effective defense mechanisms, and this project serves as a practical educational resource to illustrate the inner workings of such software.

The core features of the Keylogger include real-time keystroke capture, active window detection, structured log file management, and periodic email-based reporting. The application silently records all keyboard inputs made by the user, logging them into time-stamped files enriched with contextual metadata, including the title of the currently active window. This enables detailed analysis of user activity across different applications while intelligently handling special keys to ensure readable output.

Positioned as a valuable educational tool for cybersecurity students and professionals, the project provides hands-on exposure to concepts such as event-driven programming, daemon threading, secure data exfiltration, and executable packaging. By highlighting both the capabilities and operational methodologies of system-level monitoring software, this project imparts knowledge that is vital for developing effective security countermeasures.

# LIST OF FIGURES


1 Keylogger Project Methodology Workflow . . . . . 17

2 Keylogger Python script executing via the command-line terminal. . . . . 27

3 The Python Keylogger executable spoofed as a Google Chrome application, which silently initializes in the background upon execution without alerting the user. . . . . 27

4 System view showing the spoofed Keylogger package running discreetly as a background process. . . . . 28

5 Locally generated log file created by the Keylogger script on the target machine, containing the recorded keystroke data. . . . . 28

6 An email report delivered to the recipient's Gmail account, providing remote access to view the target system's captured logs. . . . . 29

# CONTENTS

**ACKNOWLEDGEMENT** **3**

**ABSTRACT** **4**

**List of Figures** **5**

**1 INTRODUCTION** **8**

**2 BACKGROUND** **9**

**3 THE NEED FOR A KEYLOGGER** **10**

**4 KEY FEATURES OF THE APP** **11**

**5 OBJECTIVES** **14**

**6 BRIDGING THE GAP BETWEEN TRADITIONAL SYSTEM MONITORING AND AUTOMATED KEYSTROKE LOGGING SOLUTIONS** **15**

**7 SCOPE AND DELIVERABLES** **16**

7.1 Scope . . . . . 16

7.2 Deliverables . . . . . 16

**8 METHODOLOGY** **17**

**9 PROJECT ACTIVITIES** **19**

9.1 Requirement Analysis . . . . . 19

9.2 Design & Prototyping . . . . . 19

9.3 Development . . . . . 19

9.4 Integration & Manual Testing . . . . . 19

9.5 Feedback Collection . . . . . 20

9.6 Maintenance & Future Improvements . . . . . 20

**10 RESULTS AND FINDINGS** **21**

**11 FUTURE SCOPE** **22**

**12 CONCLUSION** **24**

**13 APPENDIX** **25**

13.1 Tools and Technologies Used . . . . . 25

13.2 Resources and References . . . . . 25

13.3 Command Log . . . . . 26

13.4 Screenshots . . . . . 27

# 1. INTRODUCTION

In the modern digital landscape, cybersecurity has emerged as one of the most critical fields, with individuals and organizations facing an ever-growing range of threats. Among the various tools used in cybersecurity research and digital forensics, keyloggers hold a significant place. A keylogger is a type of surveillance software that records keystrokes made on a computer, providing valuable insight into user activity. While keyloggers are often associated with malicious use, they also serve essential purposes in ethical hacking, penetration testing, parental monitoring, and organizational security auditing.

This project presents a comprehensive, Python-based keylogger built for educational and cybersecurity research purposes. It is designed to demonstrate how keystroke logging operates at a system level, offering a practical understanding of input capture mechanisms, context tracking, log management, and automated reporting—all core concepts within the domain of information security.

By simulating the behavior of real-world surveillance software, this project serves as an educational bridge. It highlights both the stealth capabilities of monitoring tools and the risks they pose to data privacy. Ultimately, the application reinforces the need for robust security practices, endpoint protection, and informed digital citizenship by exposing how easily user inputs can be captured and transmitted.

# 2. BACKGROUND

The rapid evolution of digital technology has fundamentally transformed the way individuals, organizations, and governments interact with computing systems. As the world becomes increasingly reliant on digital platforms for communication, financial transactions, and data storage, the volume of sensitive data flowing through digital channels has intensified exponentially. Cyber threats such as data breaches, identity theft, and unauthorized surveillance continue to escalate, posing significant risks to personal privacy and organizational security.

Among the most widely studied tools in the cybersecurity domain is the keylogger. Keyloggers can capture sensitive information such as passwords, personal messages, and confidential data without the user's knowledge. They are broadly classified into two categories: hardware-based devices attached physically to a machine, and software-based programs that run in the background of an operating system. Software-based keyloggers are more commonly deployed due to their ease of distribution, stealth capabilities, and ability to transmit captured data remotely via network protocols.

Despite the abundance of theoretical resources on keylogging techniques, there is often a lack of accessible, educational implementations that demonstrate these core concepts in a practical manner. Many cybersecurity curricula rely on abstract explanations of how such tools operate, without providing students with the opportunity to build and analyze a working prototype. It is within this educational context that this project was developed, aiming to fill the gap by illustrating the full lifecycle of a surveillance application—from system-level event interception to secure data exfiltration.

# 3. THE NEED FOR A KEYLOGGER

In an increasingly digital world, understanding how keystrokes—one of the most fundamental forms of digital input—can be intercepted and potentially misused is a critical area of study in the field of information security.

While often associated with malicious intent, keyloggers serve an equally vital role as educational and diagnostic instruments. For cybersecurity students and professionals, building and analyzing a keylogger provides firsthand insight into vulnerabilities. It demonstrates how sensitive information can be silently exfiltrated, which is essential knowledge for developing effective countermeasures, intrusion detection systems, and defense strategies.

Furthermore, the project underscores the importance of endpoint security awareness. Many users remain unaware of how easily their activity can be monitored through background processes disguised as legitimate applications. By packaging the monitoring tool to mimic standard software, this project demonstrates the social engineering tactics often employed by threat actors, thereby reinforcing the critical need for vigilance and user education.

In the context of academic study, particularly within Information and Communication Technology (ICT), this project bridges theoretical knowledge and practical implementation, empowering students to think critically about detecting, preventing, and mitigating sophisticated cyber threats.

# 4. KEY FEATURES OF THE APP

1. **Real-Time Keystroke Capture:**
   - **Continuous Monitoring:** The keylogger silently captures every keystroke made by the user in real time using the `pynput` library, ensuring no input is missed.
   - **Special Key Handling:** The application recognizes and logs special keys such as Space, Enter, Tab, Backspace, Escape, and Caps Lock with readable labels (e.g., `[BACKSPACE]`, `[ESC]`), making the logs easy to interpret.
2. **Active Window Tracking:**
   - **Application-Aware Logging:** The keylogger detects and records the currently active window title using the Windows API (`ctypes`), providing context for every keystroke.
   - **Timestamped Window Headers:** Each time the user switches to a new application, a formatted header with the timestamp and window title is inserted into the log, making it easy to identify which application each set of keystrokes belongs to.
3. **Automated Log File Management:**
   - **Hourly Log Rotation:** Log files are automatically rotated every hour, creating new files with date-and-time-based naming (e.g., `2026-02-23__10-00-00.log`) for organized record-keeping.
   - **Structured Directory Storage:** All log files are neatly stored within a dedicated `logs` directory, ensuring the file system remains clean and logs are easy to locate.
4. **Automated Email Reporting:**
   - **Periodic Email Dispatch:** The application automatically emails the captured keystroke logs at configurable intervals (default: every 60 seconds) via Gmail's SMTP server, enabling remote monitoring.

- **Device-Identified Reports:** Each email report includes the hostname of the target device and a timestamp, allowing easy identification of the source when monitoring multiple systems.

5. **Stealth and Persistence:**

   - **Disguised Executable:** The keylogger is packaged as a standalone `.exe` file using PyInstaller, disguised with a Google Chrome icon and name (`Google Chrome.exe`) to avoid detection.
   - **Background Operation:** The application runs silently in the background with no visible console window or user interface, making it undetectable to the end user.

6. **Intelligent Key Filtering:**

   - **Modifier Key Exclusion:** Common modifier keys such as Shift, Ctrl, and Alt are automatically filtered out from the logs to reduce noise and produce cleaner, more readable output.
   - **Virtual Key Code Fallback:** For unrecognized keys, the application logs the virtual key code (`[VK:...]`) or an unknown marker (`[UNK:...]`), ensuring no keystroke data is lost.

7. **Secure Email Communication:**

   - **TLS-Encrypted Transmission:** All email reports are sent over a TLS-encrypted SMTP connection, ensuring that the captured data is securely transmitted and protected from interception.
   - **App-Specific Password Authentication:** The application uses Gmail's app-specific password for authentication, maintaining security without exposing the primary account password.

8. **Robust Error Handling:**

   - **Graceful Recovery:** The application includes error handling for file I/O failures, email transmission issues, and unexpected key events, ensuring continuous operation without crashes.
   - **Automatic Log File Recovery:** If the log file is unexpectedly closed due to an error, the application automatically re-opens it to resume logging without data loss.

9. **Multi-Threaded Architecture:**

- **Non-Blocking Design:** The email reporting system runs on a separate daemon thread using `threading.Timer`, ensuring that the periodic email dispatch does not interfere with real-time keystroke capture.
- **Concurrent Operations:** The keyboard listener and the email reporting module operate independently and concurrently, maximizing performance and responsiveness.

10. **Cross-Session Logging:**

    - **Persistent Logging Across Sessions:** The keylogger captures keystrokes across different application windows and user sessions, providing a comprehensive activity trail.
    - **Readable Log Format:** Logs are stored in plain-text `.log` files with UTF-8 encoding, making them human-readable and easily searchable for analysis purposes.

# 5. OBJECTIVES

The primary objective of this project is to develop a fully functional keystroke logging application designed for monitoring and recording keyboard activity on a target system. The overarching goals of the project include:

- **Comprehensive Input Capture:** To seamlessly intercept and record all user input across various applications and windows, ensuring that both standard characters and non-printable keys are accurately documented.
- **Contextual Awareness:** To integrate foreground window detection, providing analytical context by mapping captured keystrokes to specific applications or web browsers.
- **Structured Data Organization:** To implement an automated system for managing output data efficiently over long monitoring periods, preventing file bloat and ensuring chronological accuracy.
- **Remote Monitoring Capability:** To establish a secure, automated mechanism for transmitting captured data to a remote administrator, eliminating the need for physical access to the monitored machine.
- **Stealth and Resilience:** To ensure the application operates discreetly as a background process, disguised to avoid casual detection, while maintaining continuous functionality through robust error handling and non-blocking architectures.

# 6. BRIDGING THE GAP BETWEEN TRADITIONAL SYSTEM MONITORING AND AUTOMATED KEYSTROKE LOGGING SOLUTIONS

Traditional system monitoring and activity tracking methods often rely on manual observation, periodic audits, and fragmented tools that require constant human intervention. These approaches can be time-consuming, inconsistent, and prone to gaps in data collection. The Keylogger project seeks to bridge this gap by offering a lightweight, automated, and digital-first solution that operates seamlessly in the background.

The paradigm shift offered by this platform includes:

- **Passive vs. Active Monitoring:** Unlike traditional monitoring tools that may require manual activation or scheduled scans, this solution operates passively with minimal system overhead, ensuring uninterrupted data collection without affecting performance.
- **Context-Rich Data Collection:** By automatically associating inputs with the active window, raw keystroke data is transformed into meaningful intelligence, revealing workflows and user behavior rather than just random character strings.
- **Self-Sustaining Administration:** Automated file management and rotation eliminate the need for manual archiving, ensuring data remains organized and accessible without administrative upkeep.
- **Decentralized Review:** Built-in secure data transmission allows administrators to review activity logs remotely in near real-time, untethering the auditing process from the physical machine.
- **Frictionless Deployment:** Packaged as a self-contained executable, the solution circumvents complex setup procedures and software dependencies, allowing for immediate, turnkey deployment.

# 7. SCOPE AND DELIVERABLES

## 7.1 Scope

The scope of the Keylogger project encompasses the development of a background monitoring application optimized for Windows environments. The project focuses on creating a closed-loop system for data capture, organization, and exfiltration.

The scope includes real-time input capture, application-context tracking, automated file management, and secure asynchronous remote reporting. The project also covers the packaging of the software into a standalone, stealth-oriented format to simulate real-world surveillance deployment techniques. The scope deliberately excludes cross-platform compatibility (macOS/Linux) and graphical user interfaces (GUI) to maintain the software's lightweight and discreet nature.

## 7.2 Deliverables

The project delivers a fully functional, discreet monitoring tool consisting of several integrated systems:

- **Real-Time Keystroke Capture Module:** A background service that records all keyboard activity and intelligently labels special inputs.
- **Contextual Window Tracking System:** A module that inserts timestamped headers to demarcate which keystrokes correspond to which application.
- **Automated Log Management:** A chronological filing system that manages output data autonomously.
- **Secure Email Reporting Mechanism:** An asynchronous daemon that securely transmits data batches to a remote administrator.
- **Standalone Executable Package:** A compiled, zero-dependency '.exe' disguised for stealth deployment.

# 8. METHODOLOGY

The keylogger application was developed using Python, chosen for its simplicity, extensive standard library support, and wide availability of third-party packages suited for system-level interaction and automation. The methodology follows a modular design prioritizing reliability, stealth, and real-time data capture.

At the core of the application's functionality is the `pynput` library, which provides an event-driven mechanism for listening to keyboard inputs. Each key press is captured through a callback function that processes printable keys and translates special inputs into readable string formats. To capture context, the system leverages the Windows API via Python's `ctypes` module (`user32.dll`). This enables the keylogger to detect foreground window changes, logging window-switch events alongside keystrokes.

The logging mechanism relies on Python's built-in file handling and datetime libraries to implement an hourly log rotation system. This guarantees that files are kept within manageable sizes and formatted sequentially.

For the remote reporting methodology, the application utilizes the `smtplib` and `email.mime` modules to establish a secure, TLS-encrypted connection to Gmail's SMTP server. To prevent network latency from interrupting the keystroke capture loop, this reporting function is governed by Python's `threading.Timer`. By executing the email dispatch on a separate daemon thread, the core listening service remains highly responsive and non-blocking. Upon receiving these scheduled reports, the final step involves log analysis, where the formatted keystrokes and window context are reviewed to reconstruct the user's activity.

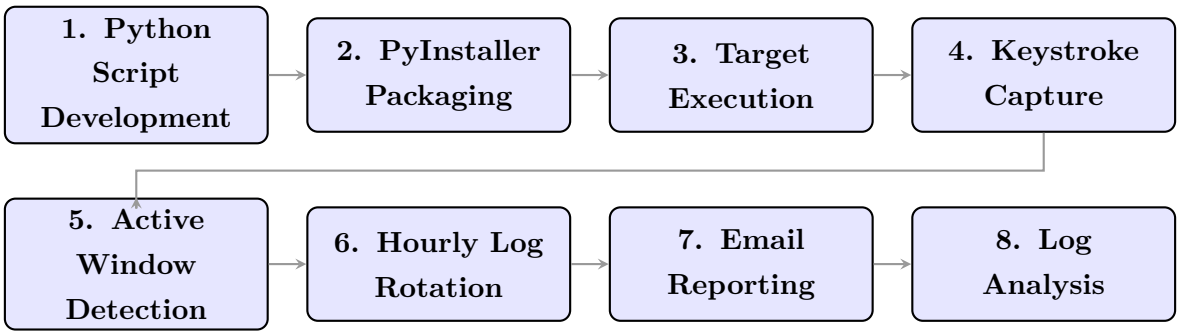


**Figure 1: Keylogger Project Methodology Workflow**

Finally, deployment methodology involved compiling the Python source code using PyInstaller. This process bundled the Python interpreter, script, and all dependencies into a single binary executable. A custom icon (`chrome-logo.ico`) was integrated during compilation to obscure the executable's true purpose, aligning with the project's educational focus on stealth techniques. Once packaged and deployed, target execution occurs silently, initiating the background monitoring processes without alerting the user.

# 9. PROJECT ACTIVITIES

The development of the keylogger application was carried out through six distinct phases, ensuring a structured approach from initial research to final deployment.

## 9.1 Requirement Analysis

In-depth research was conducted to understand the functional and technical requirements by studying existing tools, system-level input monitoring techniques, and data reporting methods. Core functionalities were established, alongside operational constraints requiring minimal system overhead.

## 9.2 Design & Prototyping

The application was designed with a modular architecture to separate concerns into distinct functional components. The data management strategy and the asynchronous, threaded approach to remote reporting were mapped out during this phase to ensure high performance.

## 9.3 Development

This phase focused on translating the architectural design into functional code. The event-capture engine was programmed, integrating intelligent key filtering. The automated delivery system was constructed to authenticate securely via TLS, and the final script was compiled into a standalone, disguised executable.

## 9.4 Integration & Manual Testing

Integration testing was critical to ensure all standalone modules worked cohesively. The listener was tested against rapid typing inputs, the window detection was validated across multiple commercial applications, and the threaded email daemon was tested under various network conditions.

## 9.5 Feedback Collection

Following the initial build, feedback was gathered to identify operational issues. Usability improvements were made to log readability, and reliability enhancements were applied to the SMTP transmission logic to handle connection timeouts.

## 9.6 Maintenance & Future Improvements

The final phase shifted focus toward the application's long-term stability. Robust error recovery was implemented for file I/O operations, and theoretical blueprints for future scaling—such as cloud integration and AI analytics—were drafted.

# 10. RESULTS AND FINDINGS

Testing and deployment of the Keylogger application demonstrated significant effectiveness in its core mission of system surveillance and data collection. The system operated with high reliability across extended monitoring sessions, successfully capturing required intelligence without degrading host machine performance.

During evaluation, the real-time keystroke logging engine performed without input lag, accurately processing both rapid typing and complex key combinations. The intelligent handling of special keys resulted in highly readable output logs, allowing reviewers to reconstruct user sentences and commands with minimal effort.

The integration of active window tracking proved to be one of the most valuable outcomes of the project. By appending context headers to the logs, reviewers were able to seamlessly map captured keystrokes to specific web browsers, word processors, or credential prompts. This elevated the raw data into actionable intelligence.

Data management tests confirmed that the hourly rotation system operated flawlessly, preventing file bloat and segmenting data into logical, chronological blocks. Furthermore, the threaded email reporting system successfully delivered these data blocks at the configured intervals. Transmissions remained secure via TLS, and the asynchronous nature of the daemon thread ensured that temporary network drops did not crash the underlying keystroke listener.

Finally, the execution of the packaged binary confirmed its stealth capabilities. The application ran discreetly without generating console windows or taskbar icons, effectively masking its presence from casual end-users while executing its monitoring directives.

# 11. FUTURE SCOPE

While the current implementation provides a robust foundation for monitoring, several enhancements can be introduced to elevate the system's capabilities, security, and versatility. The planned areas for future development include:

1. **AI Integration for Intelligent Keystroke Analysis:** Integrating artificial intelligence could allow the system to analyze captured keystroke patterns—such as typing speed and frequently used words—to provide behavioral insights. Machine learning algorithms could detect anomalies in typing behavior or flag unusual activity, transforming the keylogger into a smart, proactive threat detection system.

2. **Advanced Analytics for Deeper Insights:** Incorporating detailed data visualizations could provide administrators with graphical reports, such as keystroke frequency charts, most-used application graphs, or activity heatmaps by time of day.

3. **Multi-Platform Support:** Extending support to other operating systems, including macOS and Linux, by implementing platform-agnostic libraries for key capture and window tracking. This would make the application suitable for mixed-OS enterprise environments.

4. **Cloud-Based Log Storage and Remote Access:** Integrating cloud storage solutions (such as AWS S3 or Firebase) would allow logs to be uploaded securely in real time. Administrators could access these via a centralized web dashboard, providing a more scalable alternative to email-based reporting.

5. **Encrypted Communication and Enhanced Security:** Implementing end-to-end AES-256 encryption for local files and storing email credentials in an encrypted configuration vault would further harden the application against reverse engineering and data interception.

6. **Real-Time Remote Monitoring Dashboard:** Developing a web-based dashboard with WebSocket connections could display live keystroke feeds and active window information in real time, allowing for instant alerts and multi-device management.

7. **Screenshot and Clipboard Capture Integration:** Enhancing the tool with periodic screenshot capture and clipboard monitoring (recording copied text/URLs) would create a complete activity monitoring suite, offering visual context to the keystroke data.

# 12. CONCLUSION

The Keylogger project successfully fulfills its objective of delivering a comprehensive, background-oriented system monitoring tool. By combining real-time input capture, application-context tracking, and automated remote data transmission, the application provides a robust framework for auditing and analyzing user activity.

Through its modular design, the software achieves high reliability and performance. The use of asynchronous task scheduling ensures that data exfiltration processes do not bottleneck the core monitoring loops, while automated data management keeps historical logs highly organized and accessible. The inclusion of intelligent key filtering and window-change headers significantly improves the analytical value of the output, transforming raw keystrokes into clear, contextualized user narratives.

Furthermore, the project successfully demonstrates vital concepts in cybersecurity and digital forensics. By operating stealthily as a standalone executable, it highlights the real-world operational techniques of surveillance software, reinforcing the critical need for endpoint security and threat awareness.

While the current architecture is highly effective, the proposed integrations of cloud storage, multi-platform compatibility, and AI-driven behavioral analytics present a clear path forward. In its current state, the Keylogger stands as a highly functional, educational prototype that successfully bridges the gap between theoretical cybersecurity concepts and practical system-level programming.

# 13. APPENDIX

## 13.1 Tools and Technologies Used

The development of the keylogger script utilized the following programming tools, libraries, and frameworks:

- **Python 3:** Core programming language used to develop the keylogger script.
- **pynput Library:** Used for capturing keyboard events (key presses) in real time.
- **smtplib & email.mime (Python Standard Library):** Used for composing and sending keystroke log reports via Gmail SMTP.
- **ctypes (Python Standard Library):** Used to interface with the Windows API (user32.dll) for detecting the currently active window title.
- **threading (Python Standard Library):** Used for scheduling periodic email reports at configurable intervals.
- **PyInstaller:** Used for packaging the Python script into a standalone Windows executable (.exe) disguised as Google Chrome.
- **os, socket, datetime (Python Standard Library):** Used for file system operations, retrieving the hostname, and timestamping log entries.

## 13.2 Resources and References

The following resources and documentation were referenced during the development and troubleshooting phases:

- **ChatGPT:** For coding assistance, debugging, and understanding Python libraries.
- **YouTube:** For learning resources on Python keyloggers, pynput usage, PyInstaller packaging, and SMTP email automation.

- **Python Official Documentation (docs.python.org):** Reference for standard library modules (smtplib, ctypes, threading, os, datetime).
- **pynput Documentation (pynput.readthedocs.io):** Reference for keyboard listener implementation and key event handling.
- **PyInstaller Documentation (pyinstaller.org):** Reference for bundling the Python script into a standalone .exe with a custom icon.
- **Gmail SMTP / Google App Passwords:** Used for configuring email-based log delivery via Gmail's SMTP server (smtp.gmail.com, port 587).
- **Stack Overflow:** For troubleshooting specific issues related to Python, pynput, and PyInstaller.

## 13.3 Command Log

The following commands were utilized during the development and deployment phases of the project:

- `pip install pynput`: Installs the `pynput` library, which is the core dependency required for the application to monitor and capture real-time keyboard events.
- `python keylogger.py`: Executes the Python script directly from the terminal. This command was primarily used during the development and debugging phases to test functionality before packaging.
- `pip install pyinstaller`: Installs PyInstaller, the utility needed to bundle the Python script and all its required dependencies into a single, standalone executable.
- `pyinstaller -onefile -noconsole -name "Google Chrome" -icon=chrome-logo.ico python/keylogger.py`: Compiles the script into a standalone Windows executable (`-onefile`). It configures the application to run silently in the background without launching a visible command prompt (`-noconsole`), names the output file "Google Chrome" (`-name`), and applies a custom Chrome icon (`-icon`) to effectively disguise the executable on the target machine.

**Project Repository:**
The complete source code for this project is available at:
`https://github.com/SavioJohny/Keylogger.git`

## 13.4 Screenshots

**Figure 2: Keylogger Python script executing via the command-line terminal.**

**Figure 3: The Python Keylogger executable spoofed as a Google Chrome application, which silently initializes in the background upon execution without alerting the user.**

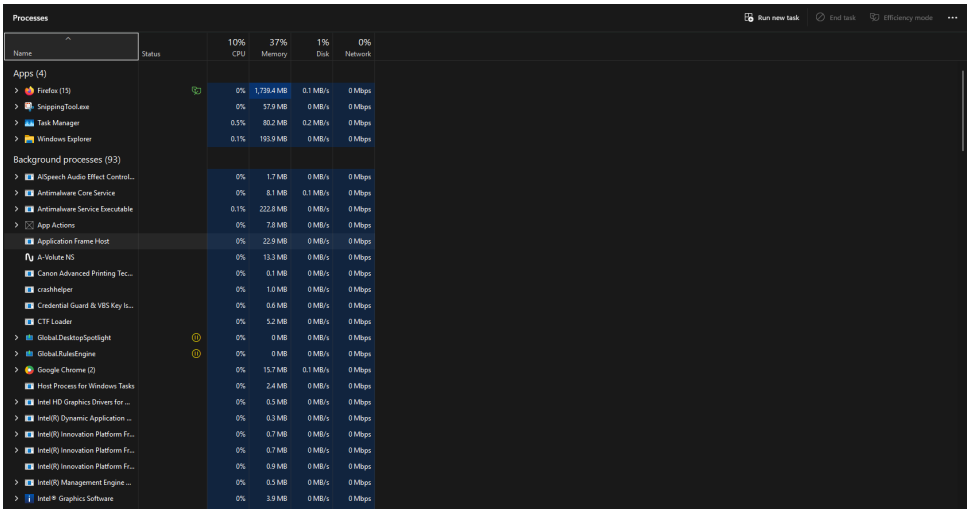


**Figure 4: System view showing the spoofed Keylogger package running discreetly as a background process.**

**Figure 5: Locally generated log file created by the Keylogger script on the target machine, containing the recorded keystroke data.**

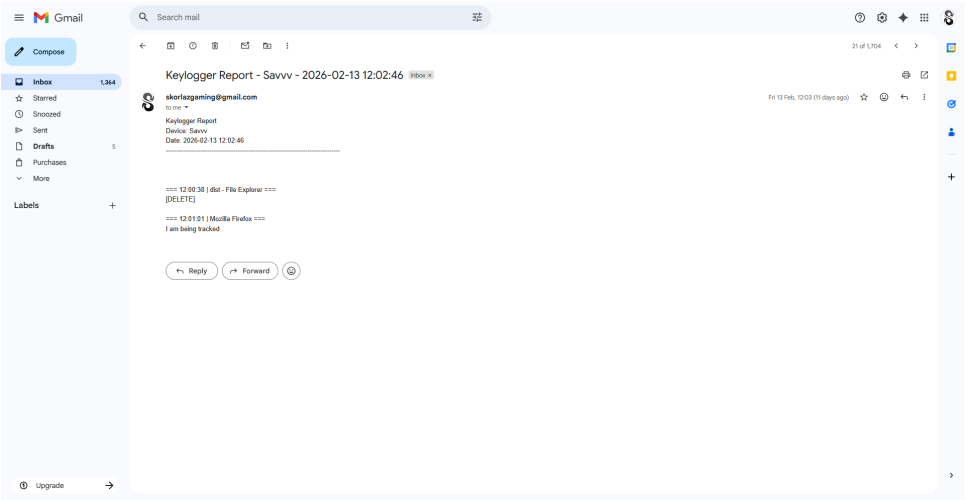


**Figure 6: An email report delivered to the recipient's Gmail account, providing remote access to view the target system's captured logs.**